

Universidade Federal de Uberlândia
Faculdade de Computação

Engenharia de Software
Prof. Fabiano Azevedo Dorça

Testes de Software
Parte 02

Testes

- **Testes Caixa-Preta e Caixa-Branca**

- Técnicas de teste podem ser classificadas como caixa-preta ou caixa-branca.
- Quando se usa uma **técnica caixa-preta**, os testes são escritos com base **apenas na interface do sistema sob testes.**
- Por exemplo, se for testar um método como uma caixa-preta, a **única informação disponível incluirá seu nome, parâmetros**, tipos e exceções de retorno.

Testes

- Quando se usa uma **técnica caixa-branca**, a escrita dos testes **considera informações sobre o código** e a estrutura do sistema sob teste.
- Por isso, técnicas de teste **caixa-preta são também chamadas de testes funcionais.**
- E técnicas **caixa-branca são chamadas de testes estruturais.**

Testes

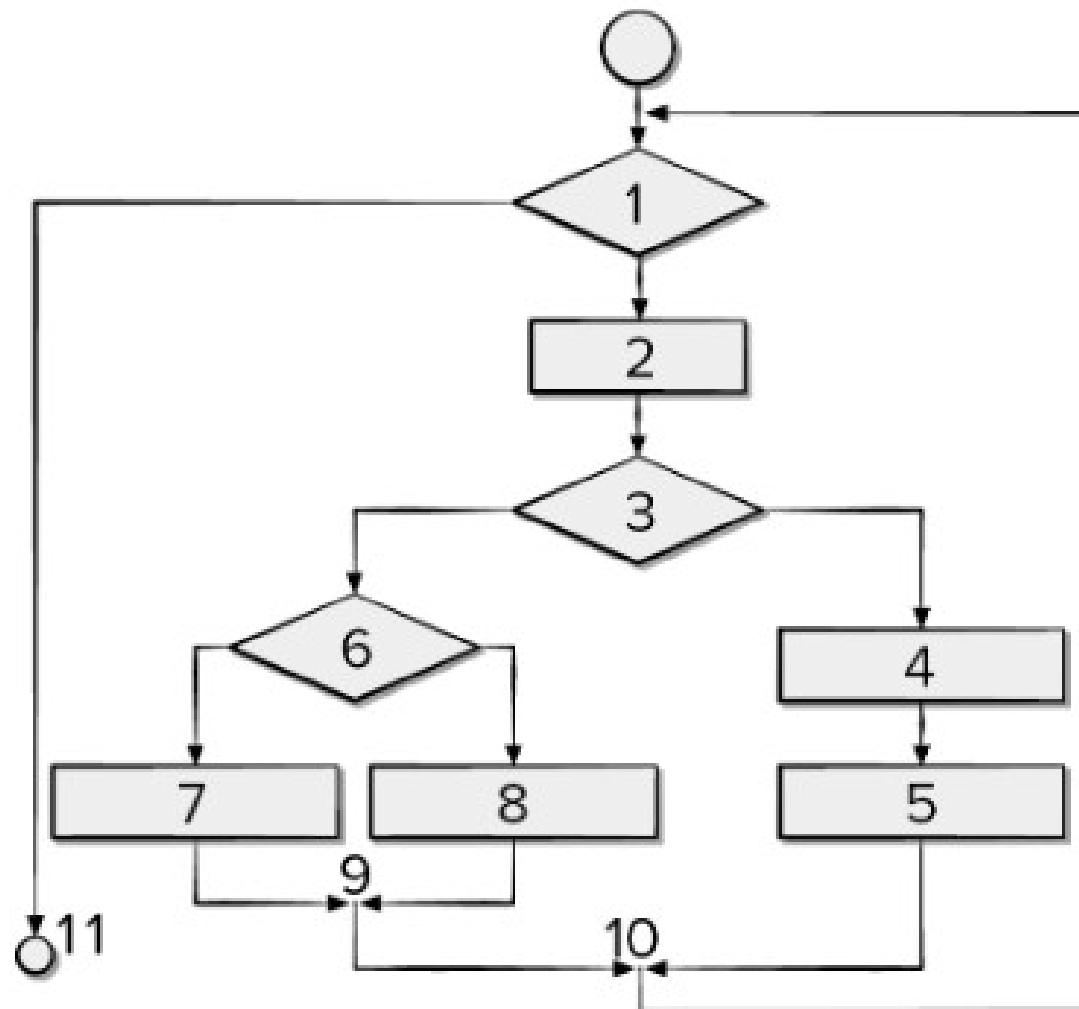
- A classificação vai depender de como os testes são escritos:
 - Se os testes de unidade forem escritos usando-se informações apenas sobre a interface dos métodos sob teste, eles são considerados como **caixa-preta**.
 - Se a escrita considerar informações sobre a cobertura dos testes, tais como desvios que são cobertos ou não, então eles são **testes caixa-branca**.

Testes

- Usando métodos de teste caixa-branca, o engenheiro de software pode criar casos de teste que
 - (1) garantam que **todos os caminhos independentes** de um módulo foram exercitados pelo menos uma vez,
 - (2) exercitem **todas as decisões lógicas** nos seus estados verdadeiro e falso,
 - (3) executem **todos os ciclos** em seus limites e dentro de suas fronteiras operacionais e
 - (4) exercitem **estruturas de dados** internas para assegurar a sua validade.

Testes

- Exemplo: Fluxograma



Testes

- Um caminho independente é qualquer caminho através do programa que introduz pelo menos um novo conjunto de comandos de processamento ou uma nova condição.
 - Caminho 1: 1-11
 - Caminho 2: 1-2-3-4-5-10-1-11
 - Caminho 3: 1-2-3-6-8-9-10-1-11
 - Caminho 4: 1-2-3-6-7-9-10-1-11

Testes

- Testes podem ser projetados para forçar a execução desses caminhos (conjunto base),
 - cada comando do programa terá sido executado com certeza pelo menos uma vez,
 - e cada condição terá sido executada em seus lados verdadeiro e falso.

Testes

- **Complexidade ciclomática**
 - é uma métrica de software que fornece uma medida quantitativa da complexidade lógica de um programa.
 - o valor calculado para a complexidade ciclomática **define o número de caminhos independentes** no conjunto base de um programa,
 - **fornecendo um limite superior para a quantidade de testes** que devem ser realizados para garantir que todos os comandos tenham sido executados pelo menos uma vez.

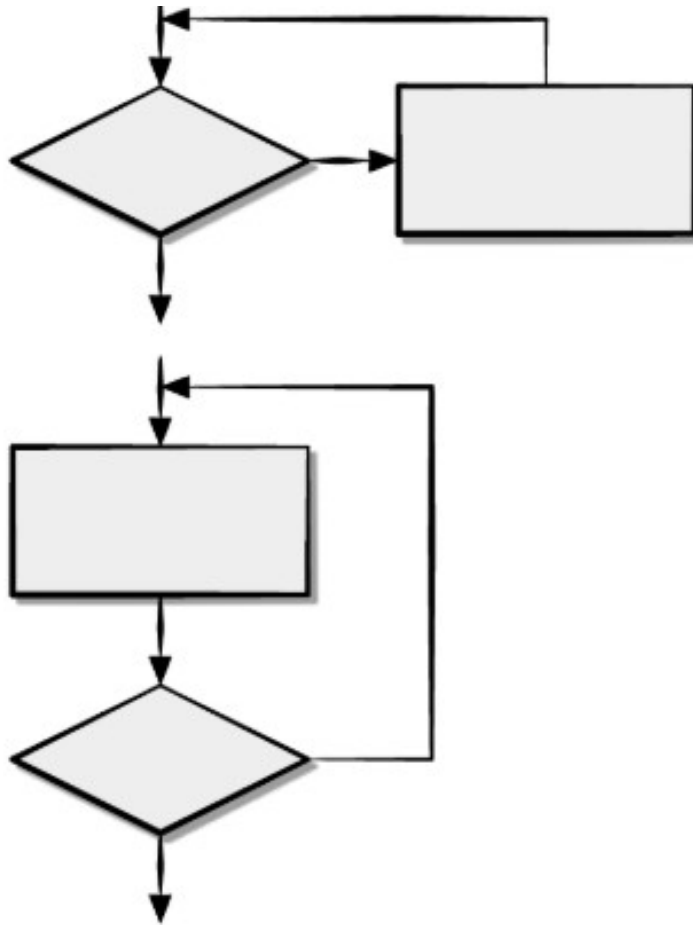
Testes

- **Teste de condição** é um método de projeto de caso de teste que exercita as condições lógicas contidas em um módulo de programa.
- O **teste de fluxo de dados** seleciona caminhos de teste de um programa de acordo com a localização de definições e usos de variáveis no programa.

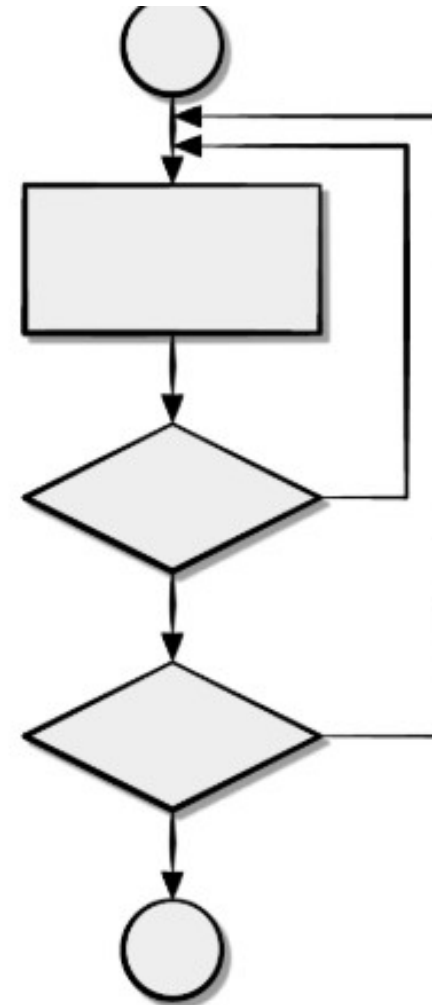
Testes

- O **teste de ciclo** é uma técnica de teste caixa-branca que se concentra exclusivamente na validade das construções de ciclo.
 - Podem ser definidas duas diferentes classes de ciclos:
 - ciclos simples e
 - ciclos aninhados.

Testes



Ciclos simples



Ciclos aninhados

Testes

- **Teste caixa-preta**,
 - também chamado de **teste comportamental** ou teste funcional, focaliza os **requisitos funcionais do software**.
 - As técnicas de teste caixa-preta permitem derivar séries de condições de entrada que utilizarão completamente todos os **requisitos funcionais para um programa**.

Testes

- O teste caixa-preta não é uma alternativa às técnicas caixa-branca.
 - Em vez disso, é uma **abordagem complementar**, com possibilidade de descobrir uma classe de erros diferente daquela obtida com métodos caixa-branca.
 - Abordagem focada no usuário.

Testes

- O teste caixa-preta tenta encontrar erros nas seguintes categorias:
 - (1) funções incorretas ou **ausentes**;
 - (2) erros de **interface**;
 - (3) erros em estruturas de dados ou **acesso a bases de dados externas**;
 - (4) erros de **comportamento ou de desempenho**;
 - Etc.

Testes

- Diferentemente do teste **caixa-branca, que é executado antecipadamente no processo de teste,** o teste **caixa-preta tende a ser aplicado durante estágios posteriores do teste.**
- Devido ao **teste caixa-preta** propositadamente desconsiderar a estrutura de controle, **a atenção é focalizada no domínio das informações.**

Testes

- O **teste caixa-preta**
 - faz referência a testes de integração realizados pelo exercício das interfaces do componente com outros componentes e com outros sistemas.
 - Um **teste caixa-preta** examina alguns aspectos fundamentais de um sistema, com pouca preocupação em relação à estrutura lógica interna do software.

Testes

- O **teste caixa-preta** se baseia em requisitos especificados nas **histórias de usuário**.
- O **teste de validação** muitas vezes **define casos de teste caixa-preta** em termos de ações de entrada visíveis para o usuário
 - e comportamentos de saída observáveis, sem **nenhum conhecimento sobre como os componentes em si foram implementados.**

Testes

- Em resumo, **testes de unidade sempre testam uma unidade pequena e isolada de código.**
 - Essa unidade pode ser **testada na forma de uma caixa-preta** (conhecendo-se apenas a sua interface e requisitos externos)
 - ou na forma de uma **caixa-branca (conhecendo-se e tirando-se proveito da sua estrutura interna,** para elaboração de testes mais efetivos).

Testes

- **Teste de caixa cinza**

- é uma técnica de teste de software que combina elementos dos testes de caixa preta e caixa branca.
- Nesses testes, o testador tem conhecimento parcial do sistema, incluindo informações internas como estrutura de dados ou código parcial, mas não tem acesso completo ao código-fonte.
- Essa abordagem permite uma visão mais ampla do sistema, **combinando a perspectiva do usuário com o conhecimento interno para identificar problemas de forma mais eficaz.**

Testes

- **Exemplo:**

- Imagine um sistema que recebe dados do usuário, processa-os e armazena em um banco de dados.
 - Um teste de caixa cinza poderia envolver o envio de dados através da interface (como um formulário web) e a verificação se os dados foram corretamente recebidos, processados e armazenados no banco de dados.
- Ao mesmo tempo, **o testador pode ter acesso a informações sobre a estrutura da tabela do banco de dados e a lógica de processamento**, permitindo uma verificação mais completa.

Testes

- O **teste de caixa cinza, nesse caso, seria mais abrangente** do que um teste de caixa preta, que só verificaria a entrada e saída do sistema, sem conhecimento interno.
- Também seria mais eficiente do que um teste de caixa branca, que poderia se concentrar apenas no código, sem levar em conta a experiência do usuário.
- Ao combinar diferentes perspectivas, é possível encontrar erros que poderiam ser ignorados em testes de caixa branca ou preta.

Testes

- **Casos de teste**

- correspondem, de certa forma a casos de uso, só que agora não mais de uso do sistema, mas com finalidades de teste.
- Assim, um caso de teste especifica um **cenário de teste**,
 - incluindo o que testar, com que entradas e com quais resultados esperados.

Testes

- **Procedimentos de teste** especificam como realizar um ou mais casos de teste.
- **Componentes de teste** automatizam um ou mais procedimentos de teste.

Esses componentes de teste podem ser desenvolvidos tanto por linguagens de script ou por meio de linguagens de programação convencional.

Testes

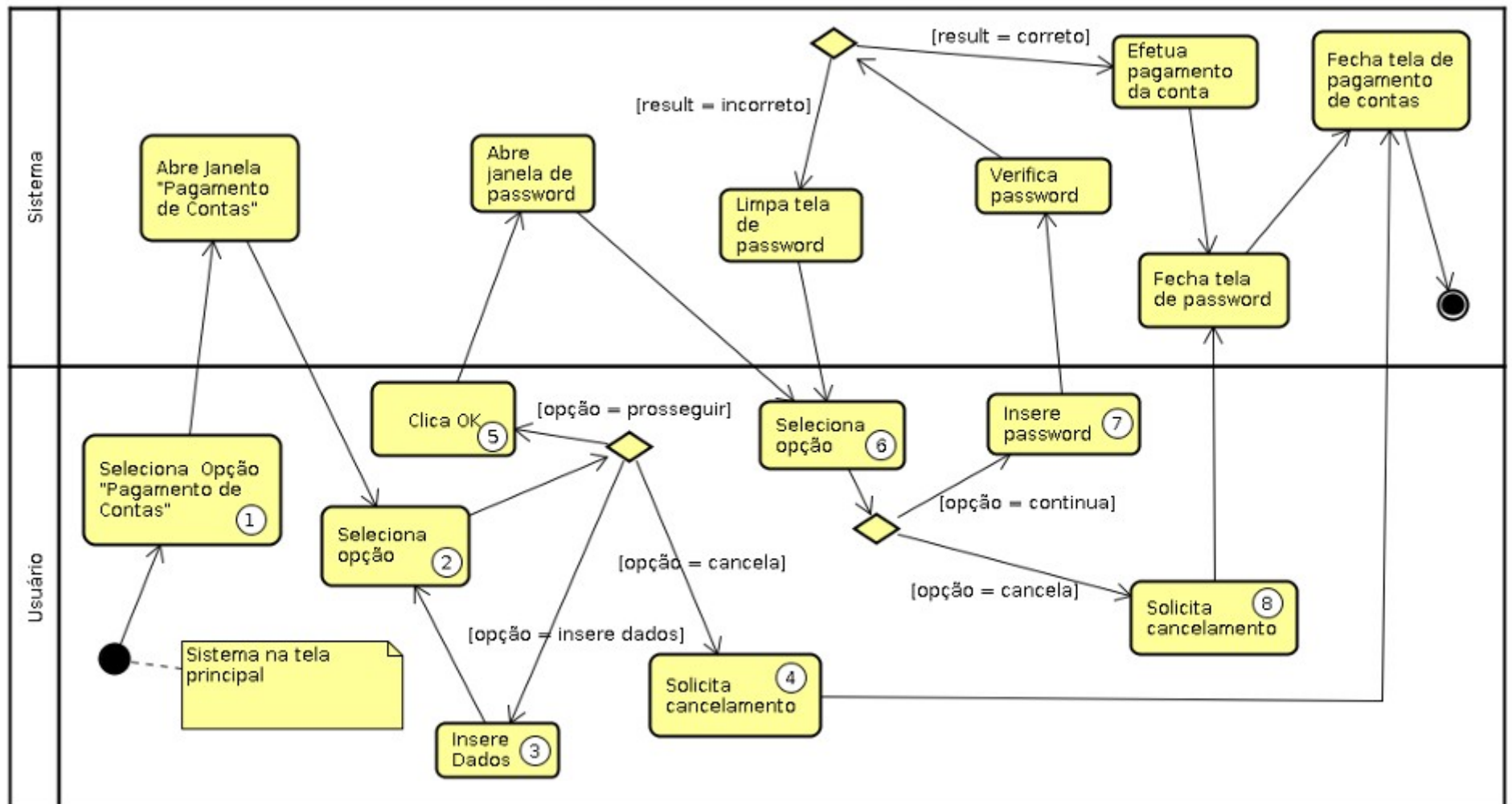
- **Design dos Testes**
 - Os objetivos básicos desta atividade são:
 - identificar e **descrever casos de teste para cada módulo**
 - identificar e **estruturar procedimentos de teste especificando como realizar os casos de teste**

Testes

- Para projetar um teste funcional, deve-se adotar o seguinte procedimento:
 - Para cada caso de uso atribuído
 - Numerar cada ação do usuário no diagrama de atividades referente ao respectivo caso de uso.
 - Gerar diferentes cenários, na forma de sequências de ações exercitando diferentes caminhos no diagrama de atividades

Testes

act Caso de Teste: Pagamento de Conta



Testes

- Observe que cada ação de parte do usuário foi numerada de 1 a 8.
- Com isso, seguindo-se o diagrama de atividades, diversos casos de teste podem ser construídos:
 - 1-2-3-2-5-6-7
 - 1-2-4
 - 1-2-3-2-4
 - 1-2-5-6-7
 - 1-2-3-2-3-2-3-2-3-2-3-2-3-2-4

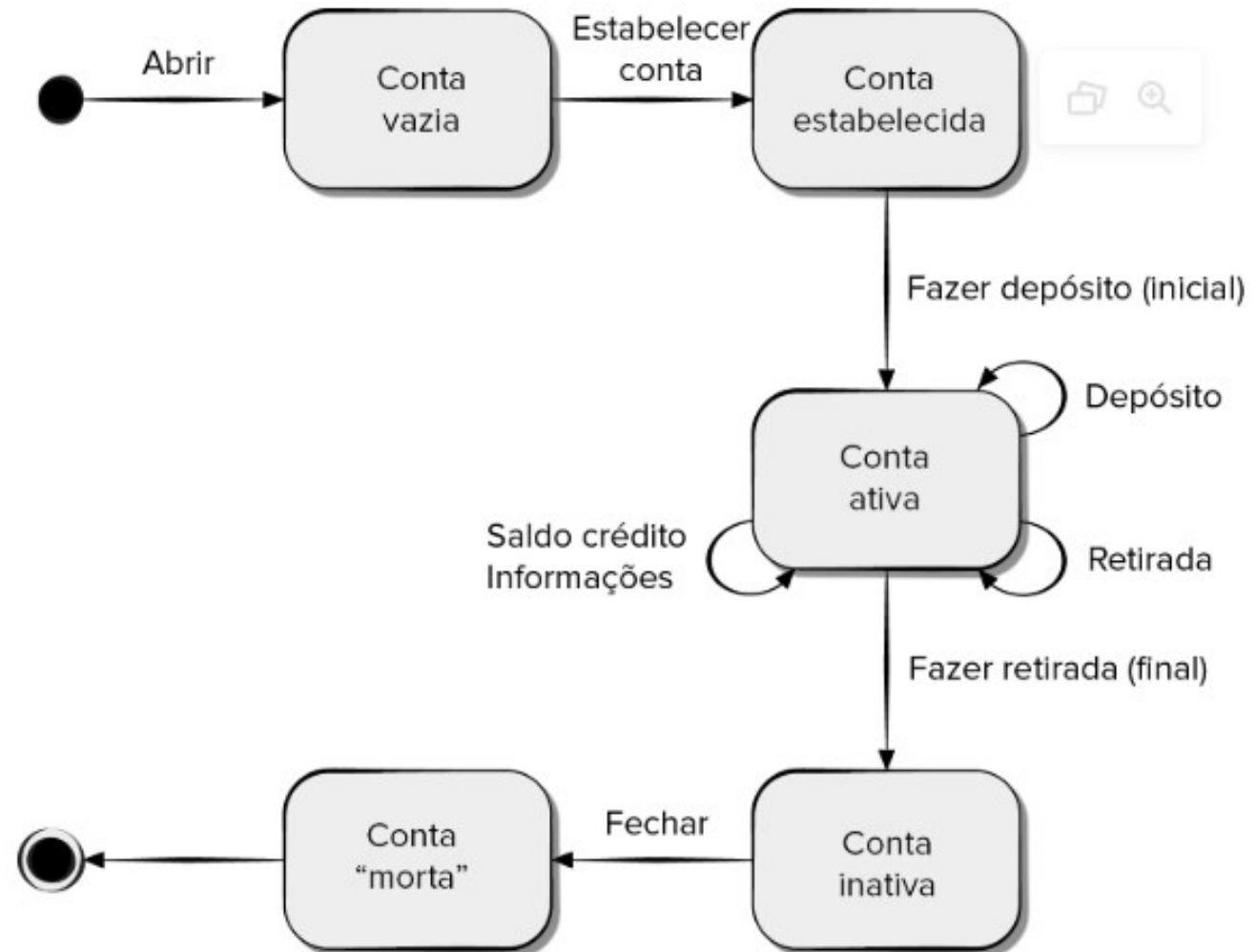
Testes

Exemplo

- O uso dos **diagramas de estado** como um modelo que representa o comportamento dinâmico de uma classe
 - O **diagrama de estado de uma classe** pode ser usado para ajudar a derivar uma sequência de testes que vão simular o **comportamento dinâmico da classe** (e as classes que colaboram com ela).

Testes

- Exemplo: Diagrama de estados para a classe Conta.



Testes

- Os testes a serem projetados **deverão cobrir todos os estados.**
 - Isto é, as **sequências de operação** deverão fazer a classe Conta fazer a transição por todos os estados permitidos:
- **Caso de teste s1:**
 - abrir → estabelecerConta → fazerDepósito (inicial) → fazerRetirada (final) → fechar

Testes

- Acrescentando as sequências de teste adicionais à sequência mínima...
- **Caso de teste s2:**
 - abrir → estabelecerConta → fazerDepósito (inicial) → fazerDepósito → obterSaldo → obterLimiteDeCrédito → fazerRetirada (final) → fechar
- **Caso de teste s3:**
 - abrir → estabelecerConta → fazerDepósito (inicial) → fazerDepósito → fazerRetirada → informaçãoDaConta → fazerRetirada (final) → fechar

Testes

- Exemplo – Documentação de caso de teste

Caso de Teste

Projeto: QualiGO

Plano de Teste: PT - Validação Geral

Suíte de Teste: Validação do módulo (Projetos)

Caso de Teste: 1		Resumo: Validar Clique no botão (Novo)	
Prioridade: Média	Status: Aprovado	Executor:	
Data de criação:			
Pré-condição: Estar logado no sistema			
Passos: 1 - Acessar o Sistema. 2 - Preencher campo (E-mail) de forma válida. 3 - Preencher campo (Senha) de forma válida. 4 - Clicar no botão (Acessar). 5 - Clicar no botão (Novo).			
Resultado Esperado: Exibir modal para cadastro de novo Projeto			
Observações: sem observações			
Comentários: sem comentários			

Testes

Caso de Teste			
CT-AUT-140: Validar cadastro de cliente			
Versão 1			
Objetivo do Teste:			
Verificar se realiza o cadastro do cliente informando nome, CPF e telefone.			
Pré-condições			
1. Usuário cadastrado e autenticado no Portal ABC; 2. Usuário com perfil Administrador; 3. Possuir CPF válido.			
	Ações do Passo	Resultados Esperados:	
1	1 - Acessar a tela de cadastro de cliente no Portal ABC: Menu principal > Cadastros > Cliente; 2 - Preencher os campos com dados válidos: - Nome - CPF - Telefone 3 - Clicar em Salvar; 4 - Verifique se o cadastro do cliente foi salvo no Banco de dados.	1 - Sistema exibe a tela de cadastro de cliente com os campos vazios; 2 - Após salvar o cadastro exibe a mensagem de sucesso: "Cliente cadastrado." 3 - O registro do cliente é salvo no Banco de dados.	

Testes

- **Especificação dos Casos de Teste**

- **Título**

- O título do caso de teste deverá ser sucinto, simples e autoexplicativo com informações para que o Analista de Teste saiba a validação a qual o teste se propõe.

Exemplos:

- Validar upload de arquivo
- Validar cadastro de usuário com perfil administrador
- Validar envio de ordem de compra

Testes

- **Objetivo**

- O objetivo do caso de teste é descrever o que será executado, fornecendo uma visão geral do teste que será realizado.
- Exemplos:
 - Verificar se realiza o upload do arquivo com as extensões permitidas
 - Verificar se o cadastro é efetivado após preencher as informações corretamente
 - Verificar se a ordem de compra é enviada informando o ativo, quantidade e preço

Testes

- **Pré-condição**

- São condições necessárias para que o caso de teste consiga ser executado.
- Exemplos:
 - Usuário cadastrado e autenticado no sistema
 - Ordem de compra enviada e executada
 - Usuário com perfil Administrador

Testes

- **Passos**

- Os passos são necessários para descrever todas as ações que o analista deve seguir durante a execução para chegar ao resultado esperado. Devendo iniciar com um verbo infinitivo (acessar, preencher, clicar, verificar) ou imperativo (acesse, preencha, clique, verifique).
- Exemplos:
 - Acessar a tela Negociação > Boleta
 - Clique no botão “Entrar”
 - Verificar se a edição foi salva no banco de dados
 - Preencha os campos do cadastro

Testes

- **Resultados Esperados**

- Descrever o comportamento esperado do sistema após executar os passos detalhados. Informar os verbos no presente (valida, apresenta, recupera, retorna).
- Evitar frases como “O sistema deve retornar a mensagem...”, prefira usar “O sistema retorna a mensagem...” para não deixar nenhuma dúvida do resultado esperado.
- Exemplos:
 - Sistema apresenta a tela de edição com os campos preenchidos.
 - A ordem é enviada e executada com o preço informado.
 - O cadastro é salvo no banco de dados.

Testes

- **Teste de Usabilidade**

- tem por objetivo **verificar a facilidade** que o software ou site possui de ser claramente compreendido e manipulado pelo usuário.
- Verifica se o sistema utiliza manuais, help on-line, assistentes eletrônicos, etc.

Testes

- **Teste de segurança**
 - O **teste de segurança é uma prática para garantir a proteção de dados e informações em sistemas**, aplicativos e redes.
 - Consiste na **avaliação de possíveis vulnerabilidades e pontos de falha** que possam ser explorados por invasores e hackers mal-intencionados, para comprometer a integridade, disponibilidade e confidencialidade dos dados.

Testes

- No teste de segurança, o analista deve **desempenhar um papel semelhante ao de um hacker**, tentando contornar todos os mecanismos de segurança implementados no mesmo.
 - As ações a experimentar são as mais diversas:
 - derrubar o sistema como um todo;
 - acessar informações confidenciais;
 - modificar informações de bases de dados;
 - interferir no funcionamento do sistema;
 - introduzir vírus de computador no sistema;
 - sobrecarregar o sistema pela multiplicação de processos executando; etc.

Testes

- A **automação de testes de segurança** permite a realização de testes de forma mais rápida e eficiente do que os testes manuais.
- Os testes manuais exigem uma equipe especializada para executar os testes, o que pode ser caro e demorado.
- Já os testes automatizados podem ser executados de forma mais rápida e eficiente, permitindo que as empresas identifiquem vulnerabilidades em um curto espaço de tempo.

Testes

- Utilizam uma variedade de técnicas e ferramentas para identificar vulnerabilidades,
 - como a análise de código-fonte, a varredura de portas e serviços, a simulação de ataques e o teste de penetração.
- A partir da análise dos resultados, é possível desenvolver estratégias de segurança mais robustas e eficazes, corrigindo as falhas e reduzindo os riscos de ataques cibernéticos.

Testes

- Quais os tipos de testes de segurança?
 - Existem diferentes tipos de testes de segurança, cada um com uma finalidade específica e voltado para diferentes áreas de segurança.
 - Principais tipos de testes de segurança:

Testes

- Teste de Penetração (Penetration Testing)
 - O teste de penetração, também conhecido como "pentest", é um **teste que simula um ataque real ao sistema ou aplicativo**, visando identificar vulnerabilidades e pontos de falha que possam ser explorados por invasores.
 - Esse teste é realizado por profissionais especializados em segurança da informação e pode ser realizado em diferentes níveis de complexidade.

Testes

- Teste de Vulnerabilidade (Vulnerability Testing)
 - É um teste que identifica as vulnerabilidades presentes no sistema ou aplicativo, sem necessariamente explorá-las.
 - Esse tipo de teste é realizado por ferramentas automatizadas ou por profissionais especializados em segurança da informação.

Testes

- Teste de Análise de Código (Code Review)
 - Consiste na revisão do código-fonte do sistema ou aplicativo, a fim de identificar vulnerabilidades e falhas de segurança presentes no código.
 - Esse tipo de teste é realizado por profissionais especializados em segurança da informação e pode ser realizado de forma manual ou automatizada.

Testes

- Teste de Segurança de Redes (Network Security Testing)
 - **Avalia a segurança da rede de computadores,** identificando possíveis vulnerabilidades e pontos de falha que possam ser explorados por invasores.
 - Esse tipo de teste pode ser realizado por ferramentas automatizadas ou por profissionais especializados em segurança da informação.

Testes

- Teste de Configuração (Configuration Testing)
 - O teste de configuração é um teste que analisa as configurações de segurança do sistema ou aplicativo, identificando possíveis vulnerabilidades e pontos de falha que possam ser explorados por invasores.
 - Esse tipo de teste pode ser realizado por ferramentas automatizadas ou por profissionais especializados em segurança da informação.

Testes

- **Teste de estresse**

- O teste de estresse, como o nome indica, consiste em verificar como o sistema vai se comportar em situações limite.
- Este limite pode ser verificado sob diferentes pontos de vista, dependendo das características do software.

Testes

- Alguns aspectos que podem ser observados neste contexto são:
 - limite em termos de quantidade de usuários conectados a um determinado sistema servidor;
 - quantidade de utilização de memória;
 - uso em diferentes versões de processadores;
 - quantidade de bloqueios encontrados num dado período de utilização; etc.

Exercícios

- 1. (ENADE 2011) Uma equipe está realizando testes com o código-fonte de um sistema. Os testes envolvem a verificação de diversos componentes individualmente, bem como das interfaces entre eles. Essa equipe está realizando testes de:
 - (A) unidade
 - (B) aceitação
 - (C) sistema e aceitação
 - (D) integração e sistema
 - (E) unidade e integração

- RESPOSTA
- LETRA E

Um software é testado para revelar erros cometidos quando projetado e construído. Uma estratégia de teste de software deve ser flexível o suficiente para atender a uma abordagem de teste personalizada e rígida o bastante para estimular o planejamento e o acompanhamento à medida que o projeto progride.

PRESSMAN, R. S.; MAXIM, B. R. **Engenharia de software**: uma abordagem profissional. 8. ed. Porto Alegre: AMGH Ed., 2016 (adaptado).

Nesse contexto, considere um sistema com as características descritas a seguir.

- Grande número de interfaces;
- Necessidade de acréscimo de módulos e possíveis modificações ao longo do desenvolvimento; e
- Grande volume de usuários e concorrência com outros sistemas que podem ser processados simultaneamente.

Para a correta produção desse sistema, a melhor estratégia de testes durante as atividades de desenvolvimento deve ser composta por testes de

- A** desempenho, unidade e caixa branca.
- B** caixa branca, esforço e integração.
- C** integração, regressão e esforço.
- D** caixa branca, desempenho e integração.
- E** recuperação, integração e segurança.

- RESPOSTA
- LETRA C

- Analise as afirmativas abaixo e dê valores Verdadeiro (V) ou Falso (F).
- () O teste de segurança é uma técnica que não avalia a resistência do software a ameaças e ataques, visando não identificar vulnerabilidades e garantir a proteção dos dados.
- () Teste de Unidade é uma técnica que verifica obrigatoriamente em todos os componentes de um software para garantir que tudo funcione conforme esperado, dispensando completamente o teste de partes menores isoladas (unidades).
- () O teste de aceitação do usuário (UAT) é conduzido exclusivamente pelos desenvolvedores para garantir que o sistema atenda aos padrões de qualidade internos da equipe de desenvolvimento.
- Assinale a alternativa que apresenta a sequência correta de cima para baixo.
- Alternativas
- (A) F - F - F
- (B) V - V - F
- (C) V - F - F
- (D) F - V - V

- RESPOSTA
- LETRA A

- Considere que um novo software foi desenvolvido e está prestes a entrar no ambiente de produção de uma empresa, mas, antes disso, serão realizados testes finais. Para isso, um conjunto de representantes dos usuários finais deve participar desse estágio de testes. Caso se perceba que o software está tendo o comportamento esperado, ele será implantado em produção.
- Qual estágio de teste está descrito no cenário acima?
- Alternativas
 - (A) Aceitação
 - (B) Componente
 - (C) Configuração
 - (D) Desempenho
 - (E) Usabilidade

- RESPOSTA
- LETRA A

- Um especialista em testes de software, com vasta experiência na criação de **testes não funcionais**, reuniu-se com sua equipe de desenvolvimento para avaliar se ainda havia requisitos não funcionais pendentes de teste em um novo software de e-commerce que estava sendo desenvolvido. Após ler a lista de requisitos identificados pela equipe como pendentes de teste, o especialista identificou um requisito não funcional ainda não testado.
- Com base no cenário apresentado, **o requisito não funcional** identificado foi o de verificar se:
- (A) a opção esqueci senha, ao ser pressionada pelo usuário, o direcionava para uma tela que solicitava o e-mail do usuário para recuperar tal senha.
- (B) o tempo de resposta para aprovar ou não alguma solicitação era de até 8 segundos.
- (C) o chatbot do sistema está apresentando o conteúdo esperado, a partir de informações fornecidas por algum usuário em um chat de ajuda acessado pelo software.
- (D) algum dos cupons de desconto de 30% para certos produtos escolhidos estava sendo aplicado corretamente.
- (E) os relatórios de venda do mês estavam sendo exportados no formato PDF e CSV.

- RESPOSTA
- LETRA B

- Quanto aos princípios fundamentais das atividades de teste e aos processos ágeis de desenvolvimento de software, julgue o item a seguir.

Os testes de regressão são realizados por ocasião da ocorrência de mudanças no software.

- Certo
- Errado

- RESPOSTA
- CERTO

- Relacione adequadamente os tipos de testes de software às suas respectivas descrições.

1. Unitário. 2. Integração. 3. Funcional. 4. Aceitação. 5. Desempenho.

() Validar se o software é aceitável para uso de acordo com os requisitos e as necessidades de negócios.

() Garantir que essas partes funcionem bem juntas como um sistema coeso.

() Verificar se o código-fonte de cada unidade funciona conforme o esperado.

() Certificar-se de que o software execute as ações esperadas e forneça os resultados corretos.

() Medir como o sistema se comporta sob diferentes condições de carga, identificando gargalos de desempenho.

A sequência está correta em:

(A) 1, 3, 2, 5, 4.

(B) 4, 5, 3, 1, 2.

(C) 3, 4, 2, 5, 1.

(D) 4, 2, 1, 3, 5.

- RESPOSTA
- LETRA D

- Ao longo do desenvolvimento de um software, os testes se fazem necessários para garantir o correto funcionamento de suas funcionalidades, os testes de software vêm ganhando cada vez mais atenção, a fim de, garantir maior qualidade e confiabilidade do produto de software a ser entregue, um dos testes que podem ser aplicados ao longo do desenvolvimento são **os testes de unidade**, Assinale a alternativa correta sobre o principal objetivo dos testes de unidade:

- (A) Testar a funcionalidade geral do software
- (B) Testar cada componente individualmente para garantir que funcionem corretamente
- (C) Testar a usabilidade do software
- (D) Testar a integração do software que está sendo desenvolvido em relação a outro software

- RESPOSTA
- LETRA B

- Sobre os testes de software, nesse tipo de teste, o objetivo é **testar a menor parte testável do sistema**, que pode ser um módulo, um objeto ou uma classe. Qual dos tipos de teste a seguir corresponde diretamente à descrição apresentada?
- (A) Teste de Integração.
- (B) Teste de Sistema.
- (C) Teste de Release.
- (D) Teste de Aceitação.
- (E) Teste Unitário.

- RESPOSTA
- LETRA E

Em razão da Covid-19, muitos estabelecimentos viram no *delivery* uma forma de manter os negócios funcionando. As mudanças de hábitos dos brasileiros revelaram um aumento do interesse nesse tipo de serviço.

Nesse contexto, um restaurante solicitou o desenvolvimento de um sistema web que possibilite gerenciar automaticamente os pedidos de entrega. O sistema foi projetado com base em tecnologias web e modelado com UML (*Unified Modeling Language*). Durante o processo de desenvolvimento foram descritos alguns itens importantes que merecem atenção no processo de teste de software.

Considerem as seguintes descrições definidas pelo analista de teste:

- 01 – Verificar se o método “Finalizar Pedido” da classe “Pedido” está funcionando corretamente.
- 02 – Garantir que o sistema funcione corretamente em diferentes navegadores de internet.
- 03 – O sistema deve passar por testes rigorosos na estrutura lógica interna do software.
- 04 – O sistema deve garantir, no mínimo, o registro de 100 pedidos simultâneos.
- 05 – O usuário deve testar o sistema em um ambiente controlado sob a supervisão dos desenvolvedores.

Considerando o texto e as descrições apresentadas, avalie as afirmações a seguir.

- I. A descrição 01 deve ser verificada com teste unitário.
- II. A descrição 02 deve ser executada com a abordagem caixa-branca.
- III. A descrição 03 deve ser executada com a abordagem caixa-preta.
- IV. A descrição 04 deve ser verificada com teste carga.
- V. A descrição 05 deve ser classificada como teste alfa.

É correto apenas o que se afirma em

- A** II e III.
- B** I, II e IV.
- C** I, III e V.
- D** I, IV e V.
- E** II, III, IV e V.

RESPOSTA

LETRA D

Um software é testado para revelar erros cometidos quando projetado e construído. Uma estratégia de teste de software deve ser flexível o suficiente para atender a uma abordagem de teste personalizada e rígida o bastante para estimular o planejamento e o acompanhamento à medida que o projeto progride.

PRESSMAN, R. S.; MAXIM, B. R. **Engenharia de software**: uma abordagem profissional. 8. ed. Porto Alegre: AMGH Ed., 2016 (adaptado).

Nesse contexto, considere um sistema com as características descritas a seguir.

- Grande número de interfaces;
- Necessidade de acréscimo de módulos e possíveis modificações ao longo do desenvolvimento; e
- Grande volume de usuários e concorrência com outros sistemas que podem ser processados simultaneamente.

Para a correta produção desse sistema, a melhor estratégia de testes durante as atividades de desenvolvimento deve ser composta por testes de

- A** desempenho, unidade e caixa branca.
- B** caixa branca, esforço e integração.
- C** integração, regressão e esforço.
- D** caixa branca, desempenho e integração.
- E** recuperação, integração e segurança.

RESPOSTA

LETRA C